



Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2023/2024

Infraestruturas Computacionais Distribuídas
Relatório: Projeto (2ª Parte)

Turma: 42D

Nome: João Ramos Número: 50730

Nome: Miguel Alcobia Número: 50746

Nome: Fábio Pestana Número: 50756

Professor: Porfírio Filipe

Data: 9 de Junho de 2024

Índice

Lista de Figuras	II
Introdução	1
Diagrama da arquitetura	2
Protocolos	2
Descrição dos componentes	3
Descrição das estruturas para persistência dos dados	4
Descrição dos protocolos ao nível da aplicação	4
Demonstração da aplicação	7
Demonstração da aplicação (Web)	9
Conclusão	14
Bibliografia	15

Lista de Figuras

Figura 1 - Diagrama da arquitetura no contexto do trabalho	2
Figura 2 - Diagrama XSD (Users)	4
Figura 3 - Diagrama XSD (Países)	5
Figura 4 - Diagrama XSD (Bandeiras)	5
Figura 5 - Diagrama XSD (Métodos Cliente)	5
Figura 6 - Diagrama XSD (Métodos Server)	6
Figura 7 - Ecrã de Login	7
Figura 8 - Ecrã de registo	7
Figura 9 - Ecrã de Visualização de um perfil	7
Figura 10 - Alteração no IP Address	8
Figura 11 - Aviso de entrada de um jogador	8
Figura 12 - Visão do Cliente com a indicação do endereço IP definido anteriormente	8
Figura 13 - Partida de Othello na visão dos dois jogadores	8
Figura 14 - Tabela de Ranking (modo consola)	9
Figura 15 - Ecrã de Login	10
Figura 16 - Ecrã de Registo	10
Figura 17 - Menu Principal da Web	11
Figura 18 - Perfil do Jogador	11
Figura 19 - Ecrã Procura Jogador	11
Figura 20 - Ecrã da Tabela de Ranking	12
Figura 21 - Ecrã para alterar cor	12
Figura 22 - Jogo Othello	12
Figura 23 - Ecrãs de Fim de Jogo	13

Introdução

Para este projeto, o objetivo é consolidar e aplicar conhecimentos sobre infraestruturas computacionais distribuídas, mais concretamente a conceção de arquiteturas e protocolos estudados em aula, envolvidos na comunicação através de uma rede de computadores. Este projeto foi dividido em duas partes, e este relatório abordará ambas as partes. Desenvolveremos uma infraestrutura computacional que possibilite dois jogadores registados jogarem Othello através dos seus respetivos browsers, dentro da mesma rede.

Temos como objetivo consolidar, através da realização deste trabalho, os conhecimentos sobre as tecnologias de desenvolvimento para a World Wide Web, utilizando JavaServer Pages (JSP) e Servlets.

Diagrama da arquitetura

Primeiramente, optou-se por uma arquitetura Thin Client, porque dado o contexto do trabalho (um jogo de Othello multijogador em rede) esta arquitetura enquadraria-se melhor para a sua resolução.

Contudo, devido a dificuldades na sua implementação, optou-se pela arquitetura peer-to-peer, embora esta segunda arquitetura apresentasse mais desvantagens.

Na segunda parte do projeto, implementou-se, como desejado de começo, a arquitetura Thin Client.

O seguinte diagrama, representa a arquitetura inserida no contexto do trabalho com detalhes respetivos às implementações realizadas na primeira parte do projeto:

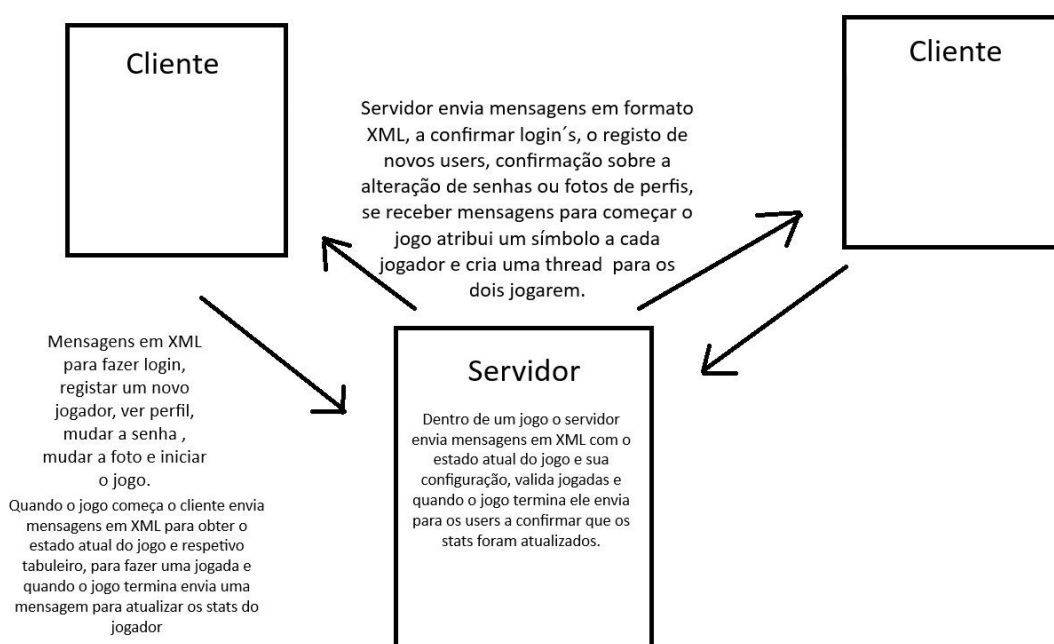


Figura 1 - Diagrama da arquitetura no contexto do trabalho

Na arquitetura Thin Client, a aplicação cliente é responsável apenas pela camada de apresentação da aplicação. Toda a lógica do sistema está centralizada no servidor (como cálculos e gestão de utilizadores, por exemplo). Desta forma, tem-se um melhor controlo e processamento dos dados no lado servidor. Além disso, os clientes podem ter poucos recursos computacionais ou não ser de confiança.

Protocolos

Os protocolos utilizados foram TCP no porto 5025, respetivamente. O servidor utilizado no trabalho tem por base um servidor TCP concorrente.

O TCP por ser um protocolo mais fiável é uma boa escolha para ser a base do servidor, pois garante a entrega, preserva a ordem de transmissão e não são aceites duplicados. Além de um processo poder

trabalhar simultaneamente com várias conexões, o que é bastante útil dado o contexto do trabalho, onde temos dois jogadores a jogar Othello em rede.

O TCP utiliza mecanismos de controlo de congestionamento que possibilitam a retransmissão de pacotes perdidos, aumentando o congestionamento da rede.

Descrição dos componentes

Fundamentalmente, podemos dividir o trabalho em três grandes partes: Servidor, Cliente e Base de Dados. Na segunda parte do trabalho, podemos acrescentar uma outra: a parte que corresponde à Web.

O servidor é responsável pela gestão do jogo e pela interação dos dois jogadores. Os jogadores à medida que entram são colocados numa lista de espera (modo FIFO, “First in - First Out”) e ao ser detetado um oponente, o servidor coloca os dois num jogo de Othello. Tanto o servidor como os clientes são implementadas com a ajuda de sockets.

A leitura e escrita de dados na socket é feita através de objetos `BufferedReader` e `PrintWriter` respetivamente, onde cada cliente tem um par destes objetos associado a ele. Cada jogo é gerido por uma `Thread`.

Esta gestão é feita a partir do envio de strings XML, que depois são transformadas em Documentos XML. Aprofundaremos o funcionamento desta abordagem ao falar dos protocolos ao nível da aplicação mais à frente.

O cliente faz o envio e a leitura de mensagens tal como o servidor, mas na perspetiva do jogador. Ele envia as mensagens das jogadas, de autenticação e todos os métodos associados à classe `User`, que gere os seus componentes: foto, username e password que são os campos sujeitos a alteração pela mão do utilizador.

A base de dados é baseada em XML e validada por ficheiros XSD. Os Users e os países (que são utilizados na atribuição da nacionalidade) são organizados num ficheiro XML e cada um tem um XSD para a respetiva validação. O ficheiro XML dos utilizadores tem toda a informação dos utilizadores: username, password (codificada em SHA256), nacionalidade, foto de perfil (codificada em base64), o número de vitórias, número de derrotas, número de jogos realizados e o tempo total gasto em partidas de Othello.

Descrição das estruturas para persistência dos dados

Para garantir a persistência dos dados usou-se ficheiros XML e XSD. Os utilizadores são todos gravados num ficheiro XML e têm um ficheiro XSD para fazer a validação do mesmo. Desta forma, garantimos a consistência e a padronização em relação aos utilizadores.

As mensagens entre os clientes e o servidor também funcionam à base de XML e da respetiva validação com um ficheiro XSD. O cliente envia uma string no formato XML para o servidor e este consulta a informação recebida como um documento XML.

No próximo tópico o protocolo no qual estas estruturas foram utilizadas será explicado com mais detalhe.

Descrição dos protocolos ao nível da aplicação

O XML foi utilizado para o envio de mensagens entre o servidor e os clientes e também para criar e guardar as informações sobre os utilizadores.

Começando pela abordagem feita nos utilizadores, estes ao serem criados são gravados num ficheiro xml (`users.xml`), tendo um nó raiz "`users`", depois cada user é representado por um nó "`user`", que tem como filhos as várias informações que dizem respeito a esse user como `username`, `password`, `nacionalidade`, data de nascimento, número de vitórias, derrotas, número de jogos realizados e, por último o tempo de jogo que o jogador já gastou em partidas de Othello.

Esta informação depois é validada por um `users.xsd` que valida estes campos e as suas devidas restrições. Por exemplo, para o `username`, criou-se o tipo simples `UsernameType` que tem como restrição o tipo de caracteres aceites e um número máximo e mínimo de letras que o `username` tem de possuir.

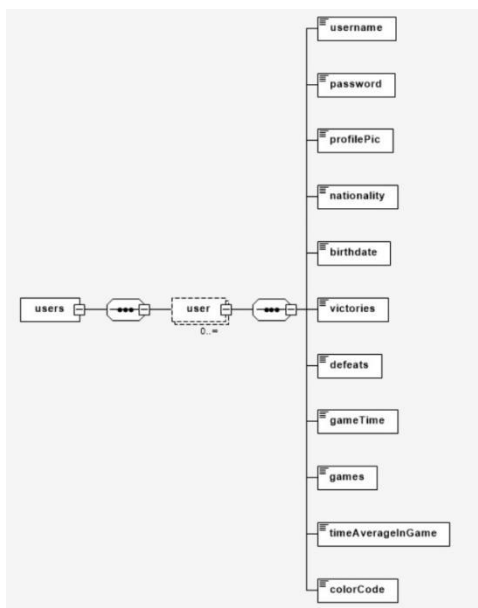


Figura 2 - Diagrama XSD (Users)

As nacionalidades também têm um xml (`países.xml`) e um xsd para validação (`países.xsd`). O seu XML tem como nó raiz “países” que tem como filhos os nós “masculinas” e “femininas”, ambos compostos por vários nós “nacionalidade”, separando assim o género das nacionalidades.

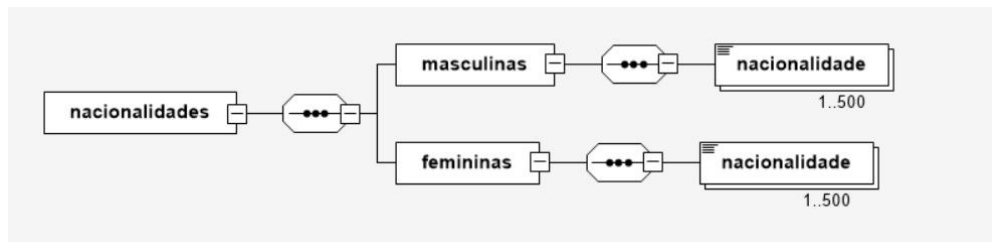


Figura 3 - Diagrama XSD (Países)

Para gerir as bandeiras das nacionalidades, criou-se um novo XML, e respetivo XSD, chamado `bandeiras.xml`. Desta forma não foi necessário modificar o XML com as nacionalidades. Este novo XML tem como nó principal “nacionalidades” e dentro dela tem várias nacionalidades que cada uma tem um “nome” e a respetiva “bandeira” codificada em Base64.

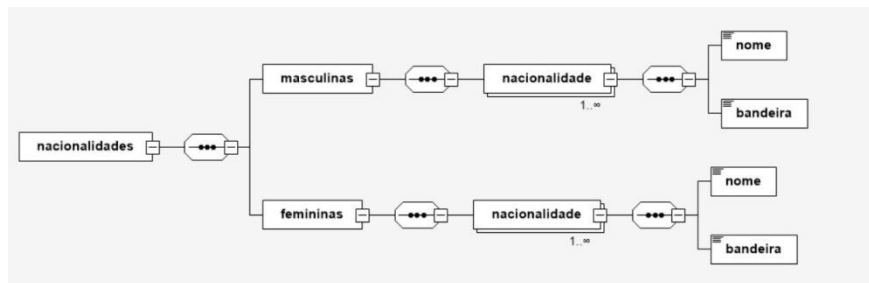


Figura 4 - Diagrama XSD (Bandeiras)

No que diz respeito às mensagens entre o servidor e o cliente, foi definido num ficheiro xsd (`othello-metodos-cli.xsd`) com o formato a ser respeitado para o envio de mensagens do servidor para o cliente no formato XML. O servidor pode enviar treze métodos diferentes para o cliente (em resposta a pedidos feitos pelo cliente):

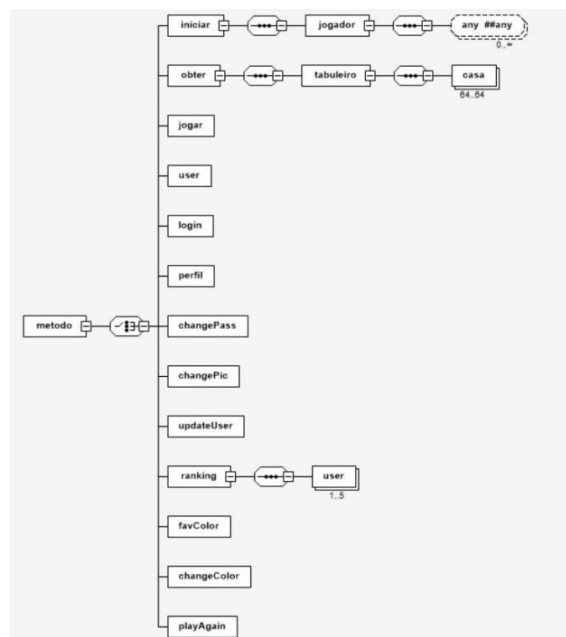


Figura 5 - Diagrama XSD (Métodos Cliente)

O formato destas mensagens é:

- Abrir a *tag* `metodo` para declarar o envio de um método;
- Abrir a *tag* do método que se pretende enviar e os respetivos atributos.

Por exemplo, o método `iniciar` tem como atributos o `username` (`nickname`) e a `password` (`senha`) do utilizador. Para enviar este método é construída a seguinte string XML:

```
<metodo><iniciar nickname=[username] senha=[password] /></metodo>
```

Quanto ao servidor, recorre a um socket para ler as mensagens enviadas pelo cliente. Transforma a string XML em um Document XML de facto e através do método `getMethod()`, ao qual se passa o Document XML e o nome do método desejado, obtém-se o corpo da função desejada. Depois basta fazer `get` do atributo que se quer.

Por exemplo, o método `runIniciar()` (que lê a informação passada pelo método `iniciar`, descrito acima, enviado pelo cliente), recebe um socket e um símbolo, que é o símbolo associado ao jogador. Com a ajuda de um objeto `BufferedReader` lê-se a informação do socket, que é a string XML.

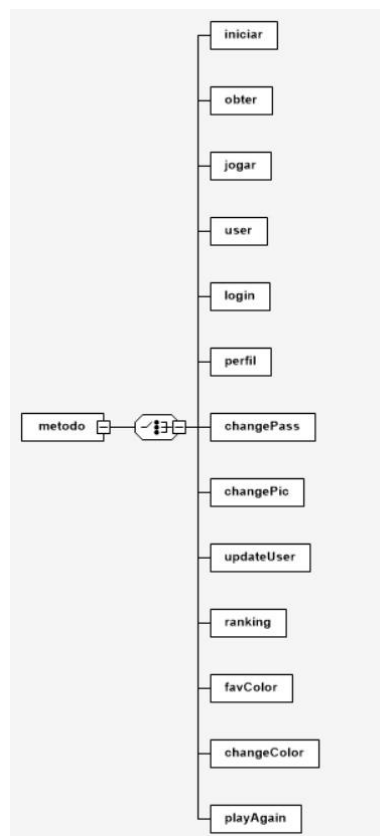


Figura 6 - Diagrama XSD (Métodos Server)

Em seguida transforma-se essa string num Document XML e chama-se o `getMethod()` com os devidos argumentos e extrai-se os atributos do `nickname` e da `senha` para variáveis distintas:

```
Document x = getNext(is);
String Nome = getMethod(x,"iniciar").getAttribute("nickname");
String Senha = getMethod(x,"iniciar").getAttribute("senha");
```

Este procedimento é usado para outros métodos, incluindo os que gerem a mecânica do jogo.

Demonstração da aplicação

Segue algumas *prints* de exemplo da aplicação.

```
*** Login ***
1 - Desejo Registrar-me?
2 - Desejo Fazer login
2
<<< ***** Indique o seu nome de utilizador:
Teste5
<<< ***** Indique a sua senha:
12345
Login com sucesso

*** Menu User ***
1 - Ver Perfil
2 - Alterar senha
3 - Alterar foto
4 - Jogar Othello
```

Figura 7 - Ecrã de Login

```
*** Login ***
1 - Desejo Registrar-me?
2 - Desejo Fazer login
1
Nome do utilizador: mikos
Senha para autenticação: pteste123
Ano de nascimento: 2004
Mês de nascimento: 01
Dia de nascimento: 21

Nacionalidade do utilizador: portuguesa
Indique o caminho para o ficheiro que contém a fotografia desejada tipo pass: player3.jpg
Imagem:
  Path = src/main/webapp/player3.jpg
  Mime = jpg
Imagem: Conteudo em memória! (45594)
Novo User criado com sucesso!
```

Figura 8 - Ecrã de registo

```
*** Menu User ***
1 - Ver Perfil
2 - Alterar senha
3 - Alterar foto
4 - Jogar Othello
1
-----
Nome do Jogador: Teste5
Data de Nascimento: 2000-12-21
Nacionalidade: portuguesa
Idade: 23
Número de vitórias: 5
Número de derrotas: 2
Número de jogos: 7
Tempo total gasto em jogos: 4 minutos, 49 segundos, 407 milissegundos.
Tempo médio por jogo jogos: 41 segundos, 343 milissegundos.
-----
```

Figura 9 - Ecrã de Visualização de um perfil

Como último colocou-se dois computadores para jogar em rede. Alterou-se o IP Address da Classe Jogador de um dos computadores para o IP do próprio PC, como mostrado abaixo:

```
IPv4 Address. . . . . : 192.168.1.81
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : fe80::ce19:a8ff:fe96:d7bf%3
                           192.168.1.254

Ethernet adapter Ligação de Rede Bluetooth:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

C:\Users\fabio>
public class Jogador {

    // **Constantes:**

    /**
     * Host padrão do servidor (endereço IP).
     */
    private final static String DEFAULT_HOST = "192.168.1.81"; //192.168.1.81

    /**
```

Figura 10 - Alteração no IP Address

```
Servidor (2) [Java Application] C:\Program Files\Java\jdk-19\bin\javaw.exe (14/04/2024, 19:51:34) [pid: 14852]
Servidor em modo de multi-game (vários jogos)!
Servidor sem timeout no acesso!
Atende o protocolo de descoberta no porto: 5026...
Servidor TCP concorrente iniciado...
Descoberta: espera por mensagens TCP no porto:5026
Aguarda ligação no porto: 5025...
Espera por um Jogador...
Descoberta: espera por mensagens UDP no porto: 5026
Aceitou ligação do Jogador: Socket[addr=192.168.1.85,port=62957,localport=5025]
Espera por um Jogador...
```

Figura 11 - Aviso de entrada de um jogador

```
Cliente -> Ligação estabelecida: Socket[addr=192.168.1.81,port=5025,localport=62957]

*** Login ***
1 - Desejo Registrar-me?
2 - Desejo Fazer login
```

Figura 12 - Visão do Cliente com a indicação do endereço IP definido anteriormente

<pre> 1 2 3 4 5 6 7 8 1 O p p p 2 X O O O p 3 p O X 4 X X X 5 X O p 6 p p 7 8 ----- Joga X primeiro linha e depois a coluna: 2 4 ----- 1 2 3 4 5 6 7 8 1 O p p 2 X O O O p 3 O O O p 4 X X O p 5 X O p 6 p 7 8 ----- Joga X primeiro linha e depois a coluna: 8 8 </pre>	<pre> 1 2 3 4 5 6 7 8 1 O 2 X O O O 3 p O X p 4 X X X 5 p X O p 6 p 7 8 ----- Joga O primeiro linha e depois a coluna: 3 5 ----- 1 2 3 4 5 6 7 8 1 O 2 X O O O 3 p O O O 4 p X X O 5 p p X X 6 p p p X 7 p 8 ----- Joga O primeiro linha e depois a coluna: </pre>
--	--

Figura 13 - Partida de Othello na visão dos dois jogadores

Para a segunda parte do trabalho criou-se mais opções no modo consola, que também serão usadas na Web, como

ver o *ranking* com os 5 melhores jogadores; fazer Logout, pedir uma “desforra” para começar um segundo jogo e, caso o pedido seja negado, os dois jogadores são redirecionados para o menu.

O *ranking* é criado através uma função `getTop5Users()` que vai ao `users.xml` que retorna os cinco melhores jogadores baseada nos seguintes critérios:

- Número de vitórias.
- Se houver empate quanto ao número de vitórias considera-se o tempo médio de jogo.

Esta função é chamada por uma outra chamada `toXMLStringForRanking()`, que estrutura a informação desses 5 jogadores em formato XML.

Para guardar as cópias de segurança, vai-se até à diretoria onde se pretende e primeiro verifica-se se já existe algum ficheiro na pasta. Caso exista compara-se a informação do *ranking* atual com a do ficheiro mais recente, que é descoberto pela data da sua última modificação, e se forem iguais não se guarda a informação; caso contrário, cria-se um novo arquivo chamado:

```
rankingTable_dd-MM-yy--HH-mm-ss-SSS-nnnnnnnnn
```

Desta forma temos a data da criação do ficheiro desde do dia até os nano segundos.

Abaixo está um *print* da tabela de *ranking* em modo consola.

```
-----Ranking-----
      Posicao: 1
Nome do Jogador: Fabio
Nacionalidade: italiano
Número de vitórias: 41
Tempo médio por jogo jogos: 21 segundos, 292 milissegundos.
      Posicao: 2
Nome do Jogador: Maria
Nacionalidade: espanhola
Número de vitórias: 34
Tempo médio por jogo jogos: 12 segundos, 35 milissegundos.
      Posicao: 3
Nome do Jogador: Miguel
Nacionalidade: portuguesa
Número de vitórias: 4
Tempo médio por jogo jogos: 44 segundos, 692 milissegundos.
      Posicao: 4
Nome do Jogador: gandalf
Nacionalidade: americana
Número de vitórias: 1
Tempo médio por jogo jogos: 5 segundos, 128 milissegundos.
      Posicao: 5
Nome do Jogador: Teste
Nacionalidade: portuguesa
Número de vitórias: 0
Tempo médio por jogo jogos:
```

Figura 14 - Tabela de Ranking (modo consola)

Demonstração da aplicação (Web)

No início da realização da segunda parte do projeto, o grupo decidiu começar pelo desenvolvimento de páginas web (HTML).

Ao contrário do modo consola, a aplicação Web dispõe de um mecanismo para pesquisar por jogadores e consultar o seu perfil, com *autocomplete*.

No mecanismo de *autocomplete* o utilizador inicia o processo ao escrever na barra de pesquisa o nome do

jogador que deseja encontrar. A cada tecla pressionada, a função JavaScript `showState()` é chamada.

Este evento é monitorizado pelo atributo `onkeyup()` do campo de texto.

Quando a função `showState()` é ativada, ela envia a string que o utilizador digitou ao servidor.

O pedido é recebido pela servlet `GetHTML` que lê os parâmetros da requisição e com os seus parâmetros, a servlet cria uma instância da classe `DummyDB` e chama o método `UserDB()` para aceder à base de dados.

Dentro do método `getData()` da classe `DummyDB`, a lista de dados é filtrada. A string de consulta é comparada com o início de cada item na lista. Apenas os itens que começam com a string de consulta.

Os resultados são formatados em HTML pelo servlet `GetHTML` e enviados de volta ao cliente. Cada item é posto dentro de uma `tag <option>`, para ser utilizada dentro de um elemento `<datalist>`.

Segue algumas *prints* dos resultados obtidos.

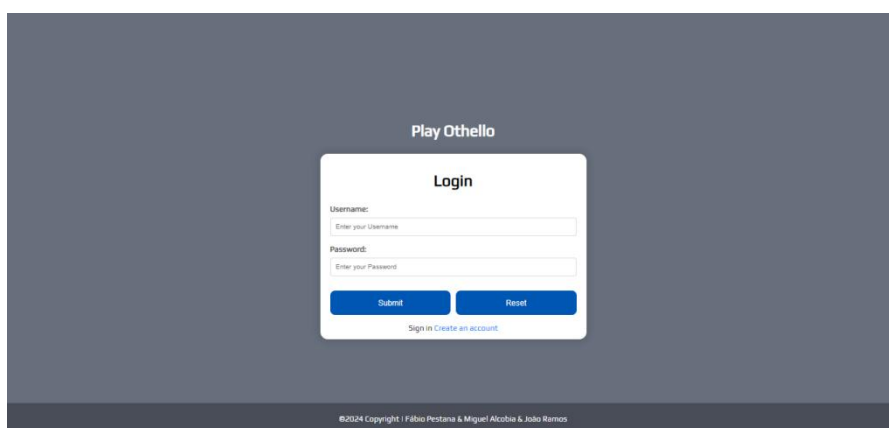


Figura 15 - Ecrã de Login

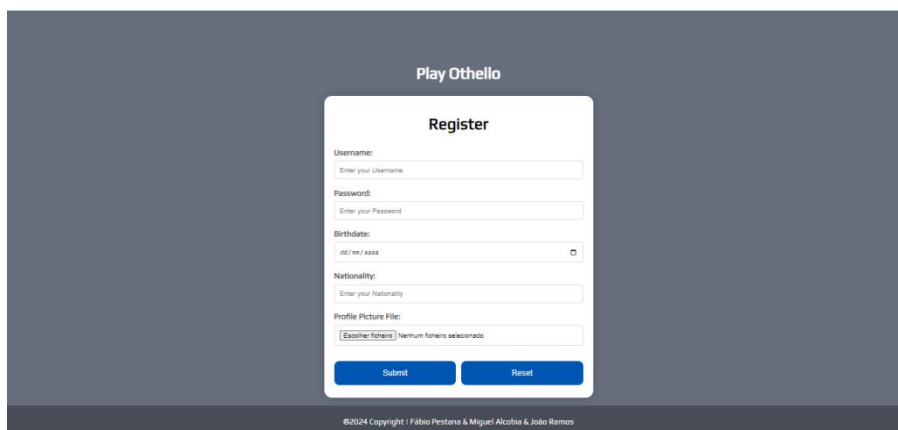


Figura 16 - Ecrã de Registo

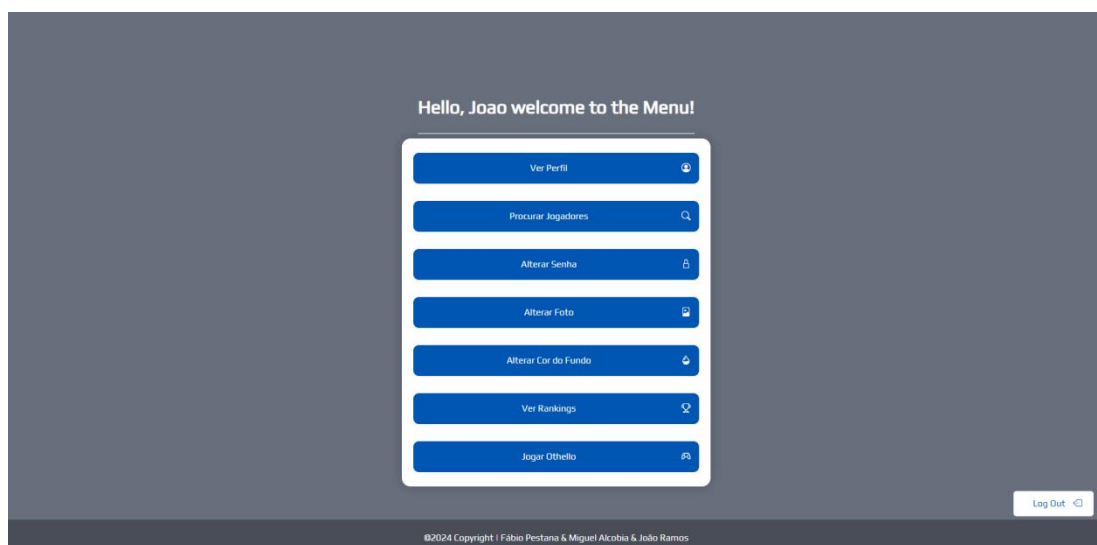


Figura 17 - Menu Principal da Web



Figura 18 - Perfil do Jogador



Figura 19 - Ecrã Procura Jogador

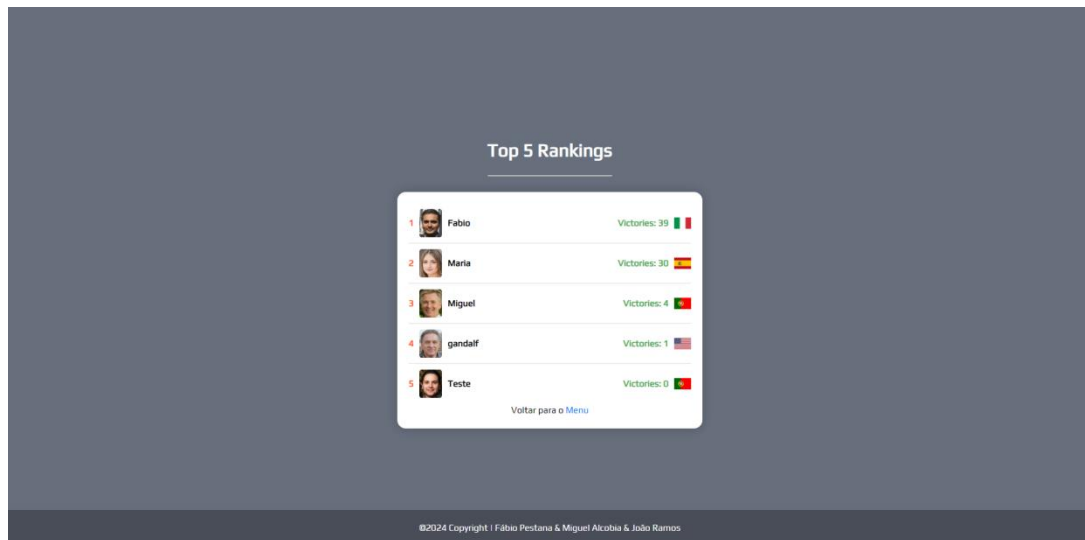


Figura 20 - Ecrã da Tabela de Ranking

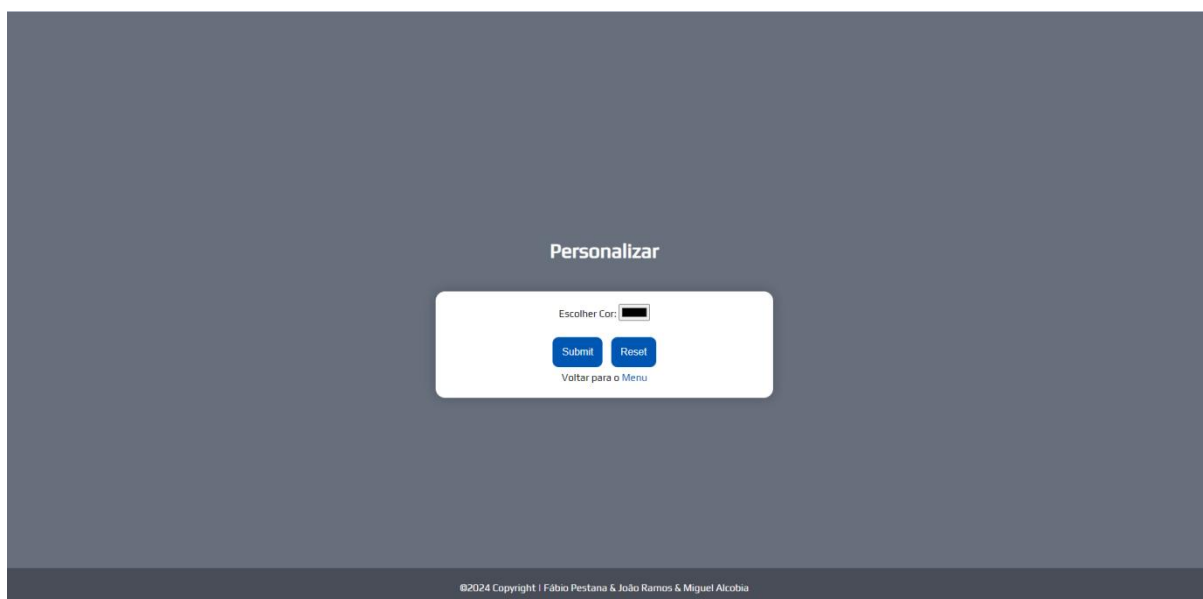


Figura 21 - Ecrã para alterar cor

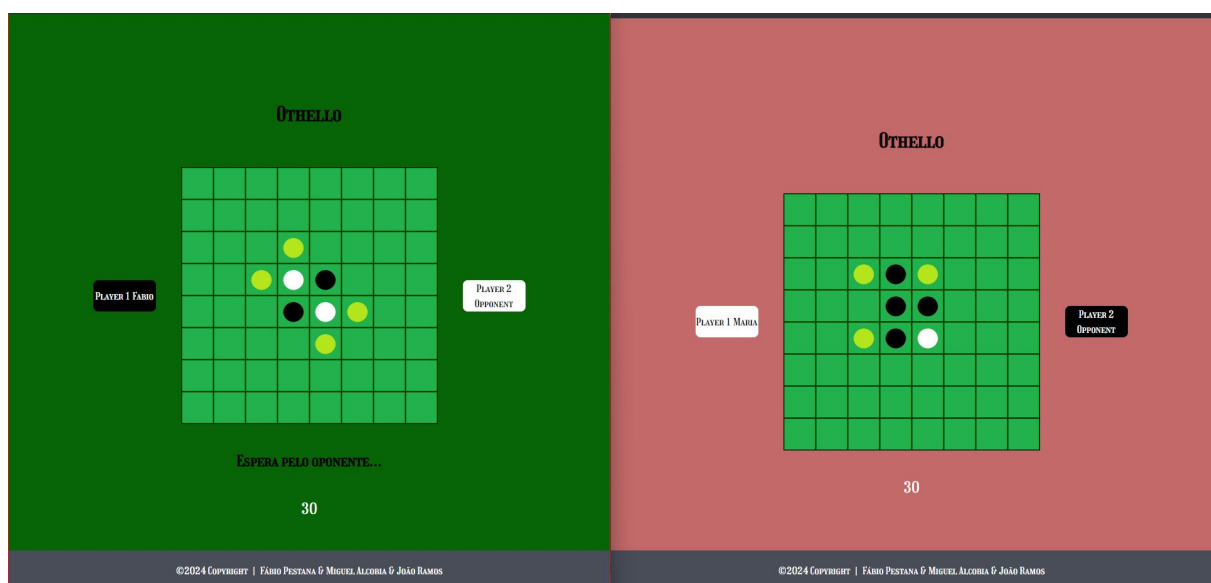


Figura 22 - Jogo Othello

As bolas verdes, correspondem às jogadas possíveis que o jogador tem disponível, que no modo consola eram

representadas por um p.

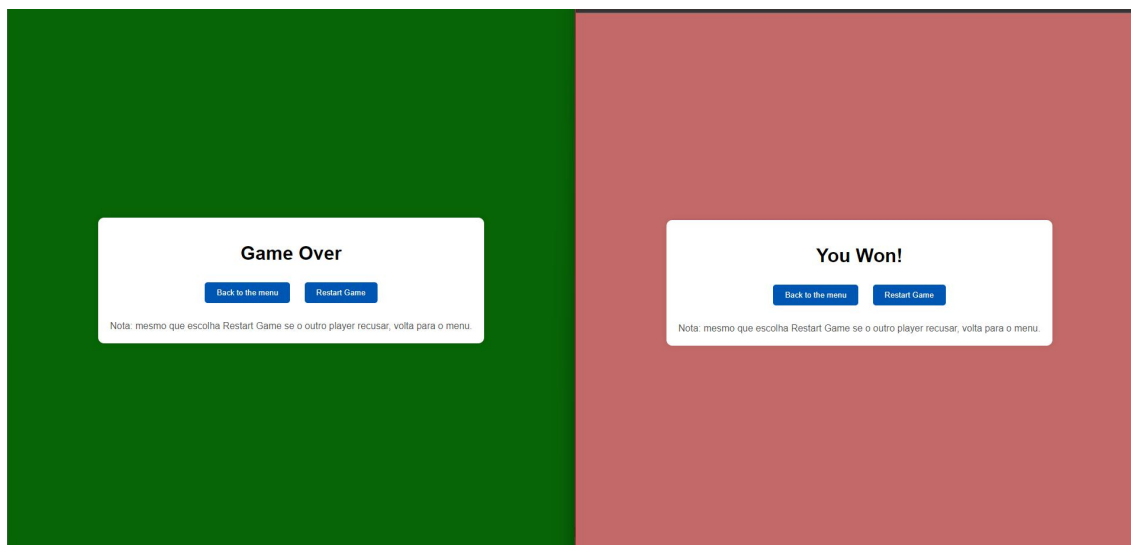


Figura 23 - Ecrãs de Fim de Jogo

Relativamente à compatibilidade entre funcionalidades utilizadas e os *browsers* mais comuns (Chrome, Safari, Firefox e Edge), analisamos o seguinte:

- `ActiveXObject`, é uma funcionalidade apenas compatível com o Microsoft Edge.
- `Date`, é uma funcionalidade com compatibilidade total com o Chrome e Edge, mas apenas uma compatibilidade parcial com os browsers Safari e Firefox, visto que suportam a data e tempo, mas não suportam o mês, semana e `datetime-local`.
- As restantes funcionalidades têm total compatibilidade com todos os browsers

Conclusão

O grupo conseguiu atingir os requisitos mínimos esperados e conseguiu corrigir falhas existentes na versão anterior do trabalho (Desformatação esporádica do XML dos utilizadores ao atualizar uma foto).

Ao testar a aplicação desenvolvida verificou-se que quando se tenta criar um utilizador com um username e a palavra-passe já existente, não existe tratamento de dados para esta situação e, portanto, surge-nos com um erro. A mesma situação ocorre para quando se pesquisa por um utilizador que não existe.

Bibliografia

Consulta

Filipe, Porfírio & Remédios Diogo. (2024) *Infraestruturas Computacionais Distribuídas*

Código:

Classes disponibilizadas, via Moodle, pelo professor Porfírio Filipe em diversos projetos WAR, foram de grande ajuda para a elaboração do trabalho.

Programa utilizado para diagramas XSD:

Cosnier, R. (2024, May 28). *dgis/xsddiagram*. GitHub. <https://github.com/dgis/xsddiagram>